

Introduction to Robotics

3st Homework

Tasks

1. The frames remain the same as Homework 1.
2. The base of the robot will be placed at $[x, y, z] = [-7.5\text{cm}, -2.5\text{cm}, 0]$ so the transformation matrix from world origin to base will be:

$$T_{wb} = \begin{bmatrix} 1 & 0 & 0 & -7.5 \\ 0 & 1 & 0 & -2.5 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

This base position resulted in the most accurate and physically correct performance of the task.

3. This frame had to change again from the previous one in order for the Robot Arm to grasp the cube from its starting position and then be capable of reaching the second pose – outside of the hole. This time the end – effector grasps the cube from its left – mid side, so the transformation matrix between the effector and the cube is the following:

$$T_{\text{eff_cube}} = \begin{bmatrix} 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & -2.5 \\ 1 & 0 & 0 & -5 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

This matrix comes from the matrix multiplication of the inverse of the world – effector transformation and the world – cube transformation

$$\text{inv}(T_{w_eff}) * T_{wc} = T_{\text{eff_cube}}.$$

The world – effector transformation (T_{w_eff}) was calculated as a rotation of the world frame about the y – axis by $\pi/2$ rads and a translation of $(x, y, z) = (22.5, 5, 2.5)$.

4. The two frames now change as follows: (see python code)

a. $T_{cube_out} = \begin{bmatrix} 0 & -1 & 0 & 10 \\ 1 & 0 & 0 & 7.5 \\ 0 & 0 & 1 & 10 \\ 0 & 0 & 0 & 1 \end{bmatrix}$, first the world frame is rotated about

z – axis by $\pi/2$ and finally translated to match the desired matrix.

b. $T_{cube_in} = \begin{bmatrix} 0 & -1 & 0 & 10 \\ 1 & 0 & 0 & 12.5 \\ 0 & 0 & 1 & 10 \\ 0 & 0 & 0 & 1 \end{bmatrix}$, it's the same transformation matrix

except a translation along y – axis, so the cube gets placed directly in the middle of the hole.

5. To design a trajectory in $SE(3)$ we need to initialize first the start and end transformation matrices and twists. So,

a. $T_s = \begin{bmatrix} 1 & 0 & 0 & 0.8 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 55.38 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

b. $V_s = [0, 0, 0, 0, 0, 0]^T$ (the joints have zero starting velocity)

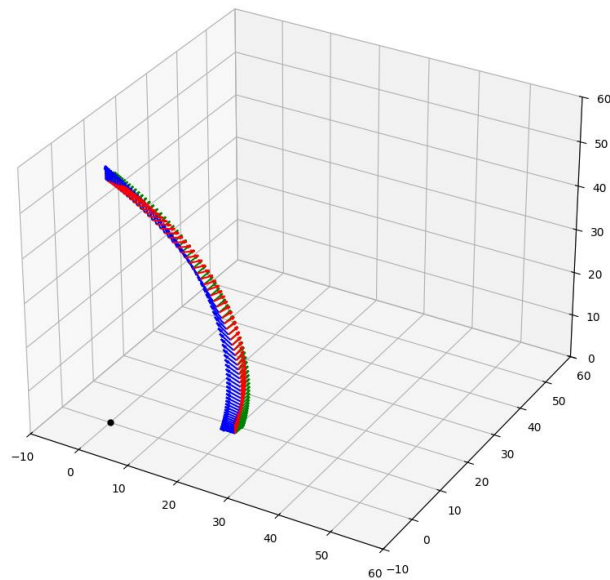
c. $T_{g1} = \begin{bmatrix} 0 & 0 & 1 & 22.5 \\ 0 & 1 & 0 & 5 \\ -1 & 0 & 0 & 2.5 \\ 0 & 0 & 0 & 1 \end{bmatrix}$ (when grasping the cube)

d. $T_{g2} = \begin{bmatrix} 1 & 0 & 0 & 17.5 \\ 0 & 0 & 1 & 7.5 \\ 0 & -1 & 0 & 10 \\ 0 & 0 & 0 & 1 \end{bmatrix}$ (when the cube is outside the hole)

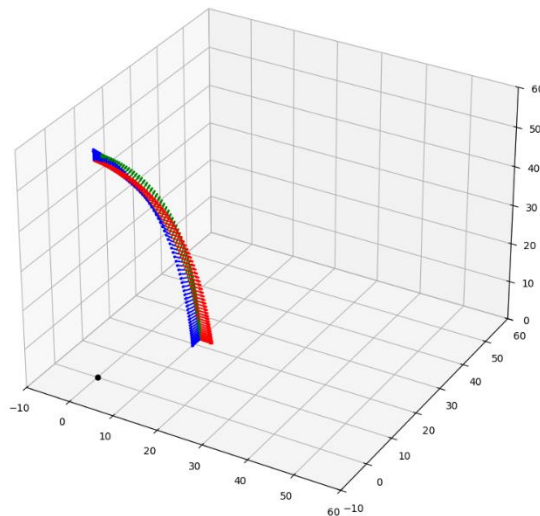
e. $V_g = [0, 0, 0, 0, 0, 0]^T$ (the joints have zero starting velocity)

Then, we have to decide the equation we are going to use as our trajectory. We pick a third-degree polynomial, cubic spline, ($c_3t^3 + c_2t^2 + c_1t + c_0$) so we have to calculate its coefficients (see *py* code *calculate_coeffs* function). We also pick a desired period to complete the task (e.g. 15 sec). Finally, we choose the time-step for our trajectory and generate the values at each time-step through interpolation.

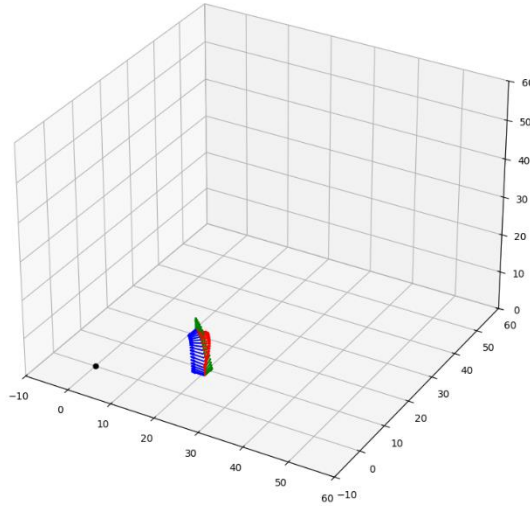
Trajectory to grasp the cube



Trajectory to place it outside the hole



Theoretical in-between trajectory



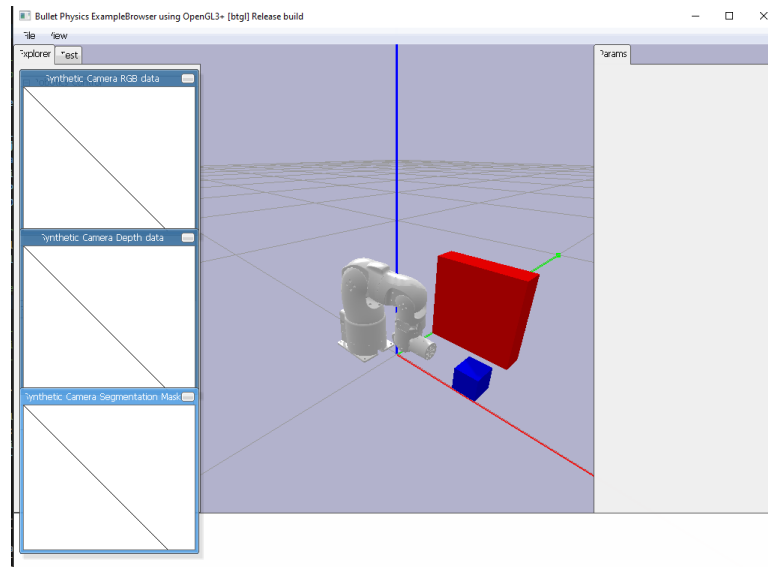
6. For the kinematic simulator we use the python library pybullet. We load the cube and the box and we place the robot arm at its starting position. We can load the angles for the robot joints to it and it will represent the task as instructed. For the generation of the joint angles to follow the designed trajectory, we use velocity commands and Euler integration (adding to angles $\dot{q} \cdot dt$), similar to the IK method, to guide the end – effector to the next frame. (Check mp4 file for full simulation).

Pybullet uses a different convention from matplotlib for 3D rotations so you may notice differences in the target matrices. For some reason, the y – axis distance between the robot's origin and the desired place for pose 2 should be $5 + 2.5 = 7.5cm$, but the distance that works in pybullet is 10cm. When the effector gets close to the cube, the grasped command turns True and using a constraint the cube stays in the same place from the effector's point of view. It was also necessary to manually turn the robot away from the red box while reaching its second pose and turn it back when it reaches that pose because the

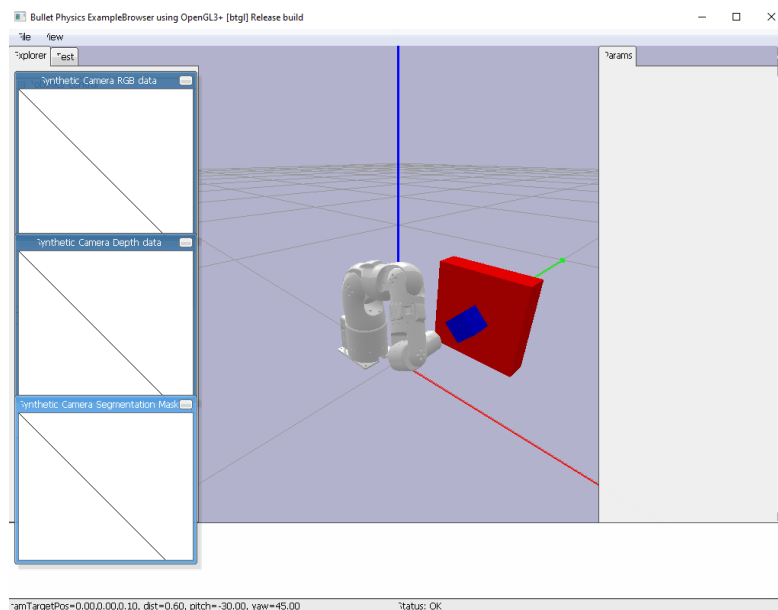
cube was colliding with the box. Finally, the last joint gets manually rotated at the end of the trajectory to match the cube to the hole's shape because if it was integrated in the target rotation matrix the robot was not reaching its desired pose.

Unfortunately, although the trajectory from pose 1 to pose 2 gets formed as shown above, the actual robot in pybullet environment cannot follow that trajectory (due to angle limits probably). So the robot gets back to its original position and then to pose 2.

Pose 1



Pose 2



7. For the task – space velocity controller we calculate $\dot{q}$ based not on the Vb twist but on the control signal $Vb_ctrl = Kp * Vb + Ki * Vb_integral$. The gains for this application were $Kp = 7.5$ and $Ki = 0.1$